

ResQGrid

Complete Phase-Wise Implementation Plan (Updated — Flutter Stack)

12 Phases • Research-Gap Mapping • Timeline • Team Split • Cost Breakdown

Table of Contents

- 1. How to Read This Plan
- 2. What Changed in This Version (v2)
- 3. Phase 1 — Planning & Setup
- 4. Phase 2 — Database Design
- 5. Phase 3 — Backend Core APIs
- 6. Phase 4 — CADERF (Evidence Reconciliation Engine)
- 7. Phase 5 — Incident Timeline + Local Knowledge Module
- 8. Phase 6 — Citizen App + ResQ Multi-Role App (Flutter)
- 9. Phase 7 — Admin/Government Dashboard
- 10. Phase 8 — Real-Time Layer (Socket.io)
- 11. Phase 9A — BLE Offline-Relay (Optional, Flutter-Native)
- 12. Phase 9B — LoRa Hardware Integration (Optional)
- 13. Phase 10 — Simulation & Evaluation
- 14. Phase 11 — Testing & Polish
- 15. Phase 12 — Documentation & Presentation Prep
- 16. Master Summary — Phase vs Research Gap Mapping
- 17. Timeline Overview
- 18. Team-Split Suggestion
- 19. Cost Breakdown

1. How to Read This Plan

Ye document ResQGrid ko banane ka poora, sequential roadmap hai — 12 core phases + 2 optional bonus-phases. Har phase mein teen cheezein clearly di gayi hain: (1) Kya banana hai, (2) Deliverable — is phase ke end mein kya working dikhna chahiye, aur (3) agar us phase mein koi Research Gap cover ho raha ho, to use orange-highlighted box mein specifically point out kiya gaya hai. Green-box mein 'Key Feature' bhi highlight kiya gaya hai jahan zaroori hai.

4 Confirmed Research Gaps: Gap 1 (Evidence Reconciliation), Gap 2 (Unified Data Model), Gap 3 (Persistent Local Knowledge), Gap 4 (Experimental Evaluation).

2. What Changed in This Version (v2)

- Tech-stack update: React Native/Expo → Flutter (isse EAS-Build wala extra-step hat gaya hai — BLE ab directly kaam karta hai).
- Phase 6 ab sirf 'Citizen + Volunteer' tak limited nahi — Hospital, NGO, Camp, aur Rescue-Team roles bhi ek single 'ResQ App' mein role-based-login se cover ho rahe hain.
- Ek naya 'Key Feature' explicitly add kiya gaya hai: Duplicate-Filtering Bottleneck (Phase 4 mein detail) — Rescue-Team tak pahunchne se pehle saare duplicate-reports hata diye jaate hain, taaki unhe sirf clean, unique incidents dikhein.

PHASE 1 — Planning & Setup (2-3 Days)

- GitHub repository banao, folder-structure decide karo (server/, resqgrid_app/)
- MongoDB Atlas account + free-cluster setup
- Node.js project init (npm init), Express install
- .env file setup (PORT, MONGO_URI, JWT_SECRET)

Deliverable: Empty project-skeleton, GitHub par pehla commit.

PHASE 2 — Database Design (3-4 Days)

- Mongoose Schemas: EvidenceReport, Incident, User (role-based: Citizen/Volunteer/Hospital/NGO/Camp/RescueTeam/Admin), LocalKnowledge, ResourceInventory
- Seeder script — dummy villages, camps, hospitals insert karne ke liye

★ **GAP 2 — EvidenceReport schema hi wo 'Unified Evidence Data Model' hai jisme Citizen/Volunteer/Hospital/Drone sabka data ek common structure (source, location, time, evidenceType, media, status) mein store hota hai.**

Deliverable: Saare models ready, MongoDB mein test-data dikhna chahiye.

PHASE 3 — Backend Core APIs (5-6 Days)

- Authentication (JWT, role-based access — 7 roles)
- CRUD APIs: /api/sos, /api/volunteer-update, /api/hospital-update, /api/ngo-inventory, /api/camp-status, /api/knowledge

Deliverable: Postman se test karke saare basic-APIs kaam karte dikhne chahiye.

PHASE 4 — CADERF — Evidence Reconciliation Engine (7-8 Days)

Ye poore project ka core research-module hai.

- Normalization function — har source ka data common-format mein convert kare
- Relationship-detection logic — Supports / Conflicts / Updates / Duplicates / Completes
- Reconciliation algorithm — multiple reports se ek Unified-Incident banane wala logic

★ **GAP 1 — DISASTER-SPECIFIC EVIDENCE RECONCILIATION** — Ye is poori phase ka core hai. Yahan hi 100+ messy reports se ek clean, trustworthy 'Incident' banta hai. Ye tumhare research-paper ka sabse important, highest-confidence contribution hai.

🔑 **KEY FEATURE — Duplicate-Filtering Bottleneck** — CADERF Engine ek 'funnel/bottleneck' ki tarah kaam karta hai. Jab bhi multiple reports (Citizen A, Citizen C, Volunteer, Drone) same incident ke baare mein aati hain, Rescue-Team ko ye SAARE raw-reports kabhi nahi dikhte. Pehle Duplicate-Detection-Step chalta hai — jo bhi reports same-location + same-content wali hain, unhe merge kar diya jaata hai. Sirf EK clean, deduplicated 'Unified Incident' Rescue-Team ke dashboard tak pahunchta hai. Isse Rescue-Team ka time waste nahi hota 100 similar-messages padhne mein — unhe seedha 'Incident #54 — 20 Victims — High Priority' jaisa clean-card milta hai.

Deliverable: 5 fake conflicting reports do, system se ek clean, deduplicated Incident nikal ke dikhao.

PHASE 5 — Incident Timeline + Local Knowledge Module (3-4 Days)

- Har Incident ke saath timestamped-history save karna
- Local-Knowledge ke 10 categories (Road, Bridge, Shelter, Boat Launch, Water Depth, Village Contact, Local Language, Medical Resource, Livestock Area, Danger Zone) ke liye API + form

★ **GAP 3 — PERSISTENT LOCAL DISASTER KNOWLEDGE REPRESENTATION** — Ye poori phase directly is gap ko address karti hai. Local-knowledge entries kabhi delete nahi hoti, sirf update hoti hain, taaki agle disaster mein outside-responders ko turant available ho.

Deliverable: Ek Incident ki timeline aur ek Village ka Local-Knowledge database mein dikhna chahiye.

PHASE 6 — Citizen App + ResQ Multi-Role App (Flutter) (8-9 Days)

Do alag Flutter-apps banenge (ya ek hi app mein role-based-routing se — team ki choice):

App 1 — Citizen App

- SOS-form (Rescue/Food/Water/Medicine/Medical-Emergency), photo-upload (image_picker), GPS-capture (geolocator)
- SOS-status tracking screen
- Nearby Camp/Hospital status view, Local-Knowledge view

App 2 — ResQ App (Role-Based, Single Codebase)

- Volunteer role: Status-update form, Local-Knowledge submission, Nearby-SOS list
- Hospital role: Beds/Doctors/Blood/Medicine/Oxygen update screen
- NGO role: Inventory-management screen (Food/Water/Blankets/Generators/Medicine)
- Camp role: Capacity + Resource-status update screen
- Rescue-Team role: Assigned-Incident list (deduplicated, clean), Local-Knowledge panel, Boat/vehicle GPS-tracking
- Backend APIs se connect (http/dio package)

Deliverable: Citizen App + ResQ App dono phone par chalte hue dikhne chahiye, real SOS bhej ke aur har role apna dashboard use karke.

PHASE 7 — Admin / Government Dashboard (React Web) (5-6 Days)

- Leaflet Map integration — SOS/Camp/Hospital markers
- Heatmap layer
- Incident-list panel (deduplicated), Resource-status panel

Deliverable: Ek working web-dashboard jo live data dikhata hai.

PHASE 8 — Real-Time Layer (Socket.io) (2-3 Days)

- Backend mein Socket.io setup
- Naya SOS/Incident bante hi dashboard automatically update ho (bina refresh)

Deliverable: Ek phone se SOS bhejo, dashboard turant update hote dikhe.

PHASE 9A — BLE Offline-Relay (Optional Bonus, Flutter-Native) (4-5 Days)

- flutter_blue_plus package integrate karo
- 'No-internet → nearby-device search → relay' logic

Flutter-Update: Ab EAS custom-dev-build wala extra-step zaroori NAHI hai — Flutter mein har app khud hi ek 'custom build' hoti hai, isliye BLE directly kaam karta hai, bina kisi extra setup ke. Cost: ₹0.

Deliverable: 2 phones ke saath live demo — internet-off karke SOS relay hote dikhao.

PHASE 9B — LoRa Hardware Integration (Optional Bonus) (3-4 Days)

- 1x ESP32 + LoRa board kharido (₹1,770-2,500), 865-867 MHz India-band
- Arduino IDE setup, LoRa library, test-code upload
- Node.js backend se serialport integration
- Dashboard par 'Received via LoRa' tag dikhana

Sirf Phase 1-8 complete hone ke baad, agar time bache tabhi shuru karo. Risk-level medium hai (hardware-issues ho sakte hain), isliye sabse last mein rakha gaya hai.

Deliverable: LoRa-module se ek test-message dashboard tak live pahunchte dikhao.

PHASE 10 — Simulation & Evaluation Module (4-5 Days)

Ye research-paper ke liye sabse zaroori phase hai.

- Dummy-data generator (1000 fake reports)
- 4 methods implement: Raw-Reports / Majority-Vote / Weighted-Fusion-Baseline / CADERF
- Comparison-results nikalo: accuracy, duplicate-rate, response-time

★ **GAP 4 — EXPERIMENTAL EVALUATION OF DISASTER EVIDENCE FRAMEWORKS** — Ye poori phase is gap ko address karti hai. Sirf 'idea batana' kaafi nahi — yahan hum numbers/simulation se prove karte hain ki CADERF baaki methods se genuinely behtar hai.

Deliverable: Ek results-table/chart jo directly research-paper mein use hoga.

PHASE 11 — Testing, Polish, Bug-Fixing (4-5 Days)

- Poore flow ka end-to-end testing
- UI polish
- Error-handling improve karna

PHASE 12 — Documentation + Presentation Prep (3-4 Days)

- README likhna (GitHub ke liye)
- Presentation PPT banana
- Demo-script practice (BLE-relay wala scene)
- Research-paper draft likhna shuru karna

Master Summary — Phase vs Research Gap Mapping

Phase	Kya Cover Hota Hai	Research Gap / Key Feature
Phase 1	Setup	—
Phase 2	Database Design	★ Gap 2 (Unified Evidence Data Model)
Phase 3	Backend APIs	—
Phase 4	CADERF Engine	★ Gap 1 + 🔑 Duplicate-Filtering Bottleneck
Phase 5	Timeline + Knowledge Module	★ Gap 3 (Persistent Local Knowledge)
Phase 6	Citizen App + ResQ Multi-Role App (Flutter)	—
Phase 7	Dashboard	—
Phase 8	Real-Time Layer	—
Phase 9A	BLE Relay (Bonus, Flutter-native)	— (Demo-value only)
Phase 9B	LoRa Hardware (Bonus)	— (Demo-value only)
Phase 10	Simulation & Evaluation	★ Gap 4 (Experimental Evaluation)
Phase 11-12	Polish + Documentation	—

Motivation Reminder: 'Policy → Software Translation' koi alag phase ya gap nahi hai — ye poore project ka overall justification hai (research-paper ke Introduction section mein).

Timeline Overview

Phase Group	Approx Days
Setup + Database (Phase 1-2)	~6 din
Backend + Core Research-Engine (Phase 3-5)	~18 din
Frontend Apps + Dashboard (Phase 6-8)	~18 din
Bonus Features + Evaluation (Phase 9A-9B, 10)	~14 din (agar dono bonus liye)
Polish + Presentation (Phase 11-12)	~8 din
Total (4-member team, parallel work)	~60-65 din (~2 mahine)

Team-Split Suggestion (4 Members)

Member	Responsibility
Member 1	Backend + CADERF Engine (Phase 2-5) — sabse research-critical role
Member 2	Citizen App + ResQ Multi-Role App (Phase 6, Flutter)

Member 3	Dashboard + GIS (Phase 7-8)
Member 4	BLE/LoRa (Phase 9A-9B) + Simulation/Evaluation (Phase 10) — research-paper-focused role

Cost Breakdown

Item	Cost
Core software (Flutter, Node, MongoDB Atlas free-tier, GitHub)	₹0
BLE integration (Flutter — no extra build-step needed)	₹0
Twilio SMS (optional)	Free-trial credits, phir paid
LoRa Hardware (1 device, optional)	₹1,770 – ₹2,500
Google Play Developer Account (optional, not needed for demo)	\$25 one-time (skip-able)
Total Minimum Realistic Budget	₹0 (core) + ₹2,000-2,500 (agar LoRa bhi karna ho)